COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	07/28/26	**DATE SUBMITTED:**	07/28/26
NAME:	Nichollo Dave A. Galvez	**INSTRUCTOR:**	Engr. Jimlord

ACTIVITY NO. HOA 4.1 [ACTIVITY NAME] Array-Based vs. STL Stack Implementations

1. PROCEDURE OUTPUT

ILO A:

HOA_4.1 > main.cpp > main()

```cpp
#include <iostream>
#include <stack> //calling stack from the STL


int main(){
    //create an object for the stack
    std::stack<int> newStack;

    std::cout <<"Stack Empty? " << newStack.empty()<<std::endl;
    //push to the stack
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout <<"Stack Empty? " << newStack.empty()<<std::endl;
    std::cout <<"THe size of the stack: " <<newStack.size()<<std::endl;
    std::cout <<"THe top of the element is: " << newStack.top()<<std::endl;
    std::cout<<"Removing elements from the stack: \n";
    newStack.pop();
    std::cout<<"The top of the element is: " <<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"The top of the element is: " <<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;

    return 0;
}
```

```
Stack Empty? 1
Stack Empty? 0
THe size of the stack: 3
THe top of the element is: 15
Removing elements from the stack:
The top of the element is: 8
The top of the element is: 3
Stack Empty? 1
```

ILO A:

```cpp
#include <iostream>


//static array, what do we need?
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;
//always -1 is the top of the stack if it is empty


//functions
void push();
void pop();
void Top();
void Display();
bool isEmpty();
bool isFull();


int main() {
   int choice; // Fixed: added missing semicolon
   std::cout << "Enter the number of max elements for the new stack: ";
   std::cin >> i;
   push();


   while(true) {
       std::cout << "stack operations" << std::endl;
       std::cout << "1.PUSH 2.POP 3.TOP 4.isEmpty 5.Display " << std::endl;
       std::cin >> choice;
       switch(choice) {
           case 1: push();
           break;
           case 2: pop();
           break;
           case 3: Top();
           break;
           case 4:
               if(isEmpty()) std::cout << "Stack is empty" << std::endl;
               else std::cout << "Stack is not empty" << std::endl;
           break;
           case 5: Display();
           break;

       }
   }


```

```
49      return 0;
50  }
51
52
53  bool isEmpty() {
54      //create a condition to check whether the top element is equal to -1
55      if (top == -1) return true;
56      return false;
57  }
58
59
60  bool isFull() {
61      //check if we reached the maximum capacity of the array
62      if (top >= (int)maxCap - 1) return true; // Fixed: prevent out of bounds
63      return false;
64  }
65
66
67  void push() {
68      //check if full -> if yes return an error
69      if (isFull()) {
70          std::cout << "stack overflow" << std::endl;
71          return;
72      }
73
74
75      std::cout << "New Value to add into the stack" << std::endl;
76      std::cin >> newData;
77
78
79      //increment the top(indext by 1) then assign the new data;
80      stack[++top] = newData;
81  }
82
83
84  void Top() {
85      //check if the stack is empty
86      if (isEmpty()) {
87          std::cout << "stack underflow" << std::endl;
88          return;
89      }
90
```

```
91
92      std::cout << "The element on the top of the stack is " << stack[top] << std::endl;
93  }
94
95
96  void pop() {
97      //check if the stack is empty
98      if (isEmpty()) {
99          std::cout << "stack underflow" << std::endl;
100         return; // Fixed: added return to stop execution
101     }
102
103
104     //display the top value
105     std::cout << "popping: " << stack[top--] << std::endl; // Fixed: added semicolon & pop logic
106     top--;
107 }
108
109 void Display(){
110     if(isEmpty()){
111         std::cout<<"Stack is empty, nothing to display. " << std::endl;
112         return;
113     }
114
115     std::cout<<"Elements in stack(top to bottom): ";
116     for(int i = top; i >= 0; i--){
117         std::cout<< stack[i] << " ";
118     }
119     std::cout<<std::endl;
120 }
```

```
Enter the number of max elements for the new stack: 5
New Value to add into the stack
3
stack operations
1.PUSH 2.POP 3.TOP 4.isEmpty 5.Display
1
New Value to add into the stack
2
stack operations
1.PUSH 2.POP 3.TOP 4.isEmpty 5.Display
1
New Value to add into the stack
1
stack operations
1.PUSH 2.POP 3.TOP 4.isEmpty 5.Display
5
Elements in stack(top to bottom): 1 2 3
stack operations
1.PUSH 2.POP 3.TOP 4.isEmpty 5.Display
```

ILO B:

```cpp
#include <iostream>
#include "stackList.h"

int main(){

    std::cout<<"--- PUSHING 'c' 'p' 'e' ONTO THE STACK --- \n";

    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    top<char>(); //Top should be 'e'

    std::cout << "\n ---- Popping all the elements ---- \n";

    while(!isEmpty<char>()){
        std::cout<<" Popped: "<<pop<char>() <<std::endl;
    }

    std::cout <<"\n ---- Testing Underflow ---- \n";
    pop<char>();

    displayAll<char>();
    return 0;
}
```

stacklist.h

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next
template <typename T>
class Node{
    public:
    T data;         //VALUE THAT WE NEED TO STORE
    Node* next;     // POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

// operations for the stack

// push
template <typename T>
void push(T newData){
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;


    //new node points to current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;

}
// pop
template <typename T>
T pop(){
    //check if the stack is empty:
    if(head<T> == nullptr){
        std::cout<< "Stack overflow. " << std::endl;
        return T();
    }

    // save the current top node
    Node<T>* temp = head<T>;

    // store its data
    T tempVal = temp -> data;

```

```
46      // move the head to the next node
47      head<T> = head<T>-> next;
48      // free the removed node
49      delete temp;
50      // return the removed value
51      return tempVal;
52  }
53
54  // isEmpty
55  template<typename T>
56  bool isEmpty(){
57      return head <T> == nullptr;
58  }
59  // top
60  template<typename T>
61  void top(){
62      // error catching:
63      if(isEmpty<T>){
64          std::cout << "Stack underflow " <<std:: endl;
65          return;
66      }
67
68      // display the Top node
69      std::cout <<"Top of Stack: "<<head<T> <<std::endl;
70
71  }
72
73  template<typename T>
74  void displayAll(){
75      if(head<T> == nullptr){
76
77      }
78      std::cout<< "Displaying the stack: \n";
79      Node<T>* temp = head<T>;
80  }
81
82  #endif
```

```
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---
Stack underflow

 ---- Popping all the elements ----
 Popped: e
 Popped: p
 Popped: c

 ---- Testing Underflow ----
Stack overflow.
Displaying the stack:
```

Conclusion:

This seatwork helped me understand how stacks work and how to implement them using arrays. I learned that arrays have a fixed size, while linked lists are dynamic and can grow until the memory crashes.

Conclusion:

2. SUPPLEMENTARY ACTIVITY

Main.cpp

```cpp
#include <iostream>
#include <string>
#include "stackList.h"

bool isMatchingPair(char open, char close) {
    if (open == '(' && close == ')') return true;
    if (open == '{' && close == '}') return true;
    if (open == '[' && close == ']') return true;
    return false;
}

// Algorithm to check if the string has balanced symbols
bool isBalanced(const std::string& expr) {
    // 1. Process character by character
    for (char ch : expr) {
        if (ch == '(' || ch == '{' || ch == '[') {
            push<char>(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {

            if (isEmpty<char>()) {
                return false;
            }

            char topChar = pop<char>();

            if (!isMatchingPair(topChar, ch)) {

                while (!isEmpty<char>()) { pop<char>(); }
                return false;
            }
        }
    }

    bool valid = isEmpty<char>();

    while (!isEmpty<char>()) {
        pop<char>();
    }

    return valid;
}

```

```cpp
int main() {
    // Test expressions from the activity sheet
    std::string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B)+[C-D]}"
    };

    std::cout << "--- BALANCED SYMBOLS CHECKER ---\n\n";

    for (const std::string& expr : expressions) {
        std::cout << "Expression: " << expr << "\n";

        if (isBalanced(expr)) {
            std::cout << "Result: VALID (Y)\n";
        } else {
            std::cout << "Result: INVALID (N)\n";
        }

        std::cout << "\n";
    }

    return 0;
```

```
Expression: (A+B)+(C-D)
Result: VALID (Y)

Expression: ((A+B)+(C-D)
Result: INVALID (N)

Expression: ((A+B)+[C-D])
Result: VALID (Y)

Expression: ((A+B)+[C-D]}
Result: INVALID (N)
```

stacklist

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

#include <iostream>

// Node structure for the linked list
template <typename T>
struct Node {
    T data;
    Node<T>* next;
};

// Global head pointer for the stack
template <typename T>
Node<T>* head = nullptr;

// Check if the stack is empty
template <typename T>
bool isEmpty() {
    return head<T> == nullptr;
}

// Push an item onto the stack
template <typename T>
void push(T newData) {
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}

// Pop an item from the stack and return its value
template <typename T>
T pop() {
    if (isEmpty<T>()) {
        std::cout << "Stack underflow.\n";
        return T();
    }
    Node<T>* temp = head<T>;
    T tempVal = temp->data;
    head<T> = head<T>->next;
    delete temp;
    return tempVal;
}

```

```
46  // Get the top item without removing it
47  template <typename T>
48  void top() {
49      if (isEmpty<T>()) {
50          std::cout << "Stack underflow\n";
51          return;
52      }
53      std::cout << "Top of stack: " << head<T>->data << "\n";
54  }
55
56  // Display all items in the stack from top to bottom
57  template <typename T>
58  void displayAll() {
59      if (isEmpty<T>()) {
60          std::cout << "Stack is empty\n";
61          return;
62      }
63      std::cout << "Stack contents (top to bottom): ";
64      Node<T>* temp = head<T>;
65      while (temp != nullptr) {
66          std::cout << "\n| " << temp->data << " |";
67          temp = temp->next;
68      }
69      std::cout << "\n";
70  }
71
72  #endif
```

Expression	Valid? Y/N	Output	Analysis
(A+B)+(C-D)	Y	`Expression: (A+B)+(C-D)` `Result: VALID (Y)`	All opening parentheses (have matching closing) in the correct order.
((A+B)+(C-D)	N	`Expression: ((A+B)+(C-D)` `Result: INVALID (N)`	Missing a closing) at the end. The stack is not empty when processing finishes.
((A+B)+[C-D])	Y	`Expression: ((A+B)+[C-D])` `Result: VALID (Y)`	All parenthetical () and square bracket [] pairs are correctly balanced nested.

((A+B)+[C-D]}	N	Expression: ((A+B)+[C-D]} Result: INVALID (N)	Mismatched closing delimiter: opened with (but closed with }

Summary of Lessons Learned

Through this activity, I implemented a dynamic, linked-list-based stack (stackList.h) and practiced essential operations (push, pop, top, isEmpty). I then applied these functions to solve a real-world problem: validating balanced brackets in math expressions. Using a linked list ensured the stack expanded dynamically without fixed size limits.

Analysis of the Supplementary Activity

The program processes expressions character by character, pushing opening brackets onto the stack and popping them when closing brackets appear to check for valid pairs. If a mismatch occurs or a closing bracket is found on an empty stack, the expression is flagged as invalid. The expression is confirmed valid only if the stack is entirely empty after reading the whole string.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini and grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.